Topic 4: Agile Design Flows

ECE 9433: Agile System-on-Chip Design

Spring 2024

Prof. Austin Rovinski

New York University

Let’s Start from the Basics

- **Flow**: a sequential process of developing from one state to another
- We want to develop an *idea* into *prototype*
  - In the scope of this class: RTL to GDS (graphical Database System).
- First, let’s understand different methodologies behind flows before moving into specifics

Contrasting Hardware Design Philosophies

Waterfall VS **Agile**

- Careful planning in advance
- Design specifications and stick to them
- Ex: Civil engineer building a dam
  - Carefully design and model dam
  - Build and test molds
  - Can only pour concrete once

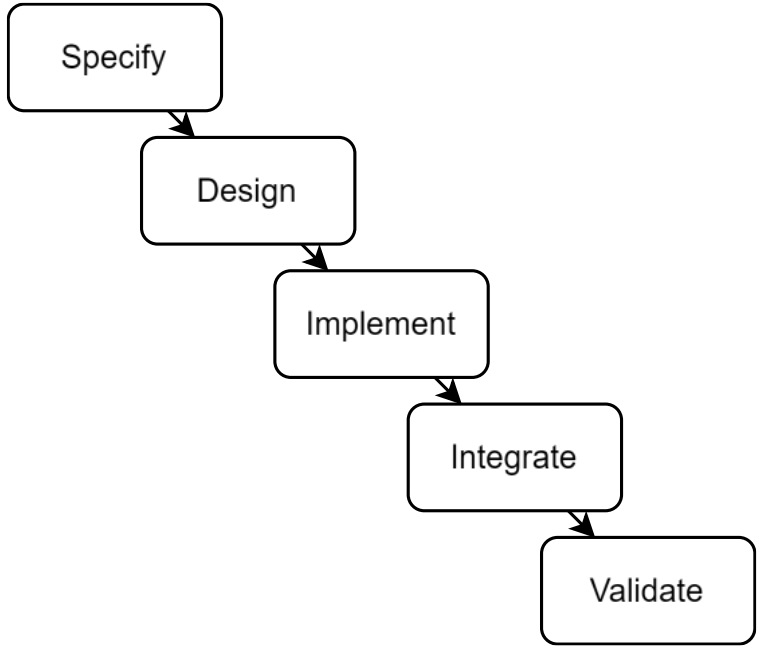
- Can’t know perfect design in advance
- Reassess context and adapt
- Ex: Competitive athlete
  - Prepare, but be ready to adapt
  - How could you know what your opponent will do in advance?

Adapted from S. Beamer

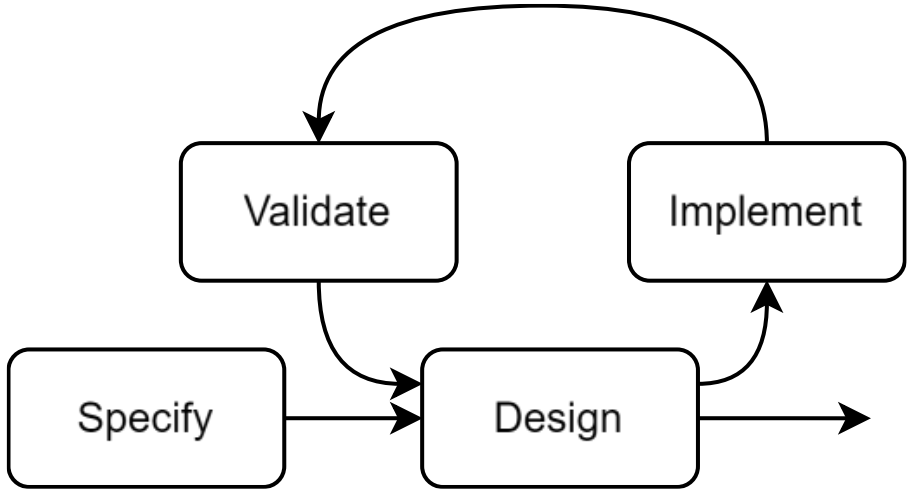
Contrasting Hardware Design Philosophies

Waterfall VS **Agile** (VIP - minimum viable product)

- Complete before moving to next step
- Late integration results in need to optimize and validate again



- Integrate and verify design early
- Incrementally add features and optimize
- Iterate as much as necessary



Adapted from S. Beamer

Enabling Insights of Agile Techniques

- Recognize end goal (or at least path to it) will be hard to predict
  - Were predicted constraints accurate?
  - How sensitive is the design to changes?
- Agile techniques already proven successful and used in software
- Viewing hardware design as fluid enables Agile techniques
  - Recognize that you can’t perfectly predict constraints and optimizations

Adapted from S. Beamer

Potential for Agile Techniques for HW Design

- *Reduced design effort*
  - Don't waste effort on unnecessary components or optimization
  - Identify risk factors early → go through whole flow ASAP.
  - Ex: finishing step requires fiducial insertion which modifies your floorplan
- *Improved design result* (pre-optimization avoided).
  - Can better allocate effort for optimization
  - Effectively profile-guided optimization
- *Increased predictability of design process*
  - Fewer surprises at integration stage; components interacting from the start

Adapted from S. Beamer

Q: Is Waterfall Ever Useful?

→ chip design use simulator.

- Agile is good when:
  - Trial and error are "inexpensive"
  - Trials provide meaningful feedback → timing, DRC.
- Simulating a chip happens in software – relatively cheap
- Failed implementation = retry with new parameters
- Waterfall is better when trial and error are expensive!
  - Construction
  - Safety-critical activities
  - Strict regulations

Design Strategies for Agile Hardware Design

- Reuse
- Modularize
- Automate

Reuse

- Don’t reinvent the wheel!
- Open-source hardware is thriving. See if a solution already exists
  - Always check to see if it satisfies your needs
  - If it doesn’t, estimate how much time to adjust it vs. doing it yourself
  - Same goes for closed-source IP, but factor in cost
- Reuse open-source frameworks, testing, IP, …

github, opencourse.

Don’t try to force existing solutions that won’t work

Reuse

Build sth on top of it

(integrating IP is very common).

https://github.com/aolofsson/awesome-opensource-hardware

Designs & Generators

Accelerators

- aes
  - Symmetric block cipher AES (Advanced Encryption Standard)
- ara
  - Vector Unit, compatible with the RISC-V Vector Extension
- bfg
  - Compiler for Reduced-Complexity Reconfigurable Fabrics
- bismp
  - Chisel-based bit-serial matrix multiplication accelerator generator
- finn
  - Quantized NN to FPGA dataflow accelerator generator
- fftgenerator
  - MMIO-Based FFT Generator
- fpu
  - Synthesizable ieee 754 floating point library in verilog
- garnet
  - CGRA generator
- gemmini
  - Berkeley Spatial Array Generator
- gplgpu
  - GPL v3 2D/3D graphics engine in verilog
- core_jpeg
  - High throughput JPEG decoder in Verilog for FPGA
- fftgenerator
  - Chisel based FFT generator

CPUs

- a2i
  - A2I POWER processor core RTL (VHDL)
- ara
  - 64-bit Vector unit coprocessor to Ccva6
- black-parrot
  - Linux-capable RISC-V multicore
- cfu-playground
  - Famework for playing with custom opcodes to accelerate TensorFlow Lite for Microcontrollers
- cores-swerv
  - SweRV EH1 RISC-Vcore
- cores-swerv-el2
  - SweRV EL2 RISC-V Core
- core-v-verif
  - Functional verification project for the CORE-V family of RISC-V cores
- cva6
  - Linux capable RISC-V CPU
- cve2
  - Small two-stage 32 bit RISC-V CPU core (RV32IMC/EMC)
- cv32e40s
  - RV32IMFCX RISC-V 4-stage secure RISC-V CPU
- cv32e40x
  - RV32IMFCX RISC-V 4-stage compute RISC-V CPU
- cvw
  - Configurable RISC-V Processor for RISC-V System-on-Chip Design textbook.
- ibex
  - Small 32 bit RISC-V CPU core

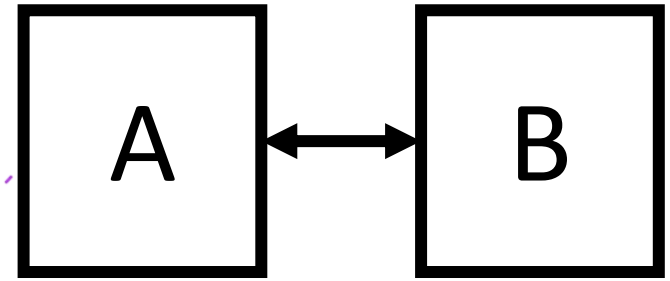
Dangers of Open-Source

Known chip design firm -
validation. Kits/results.
documentation.

- "Wild west" of quality
  - You need to do your own research to see how thoroughly validated the IP is
  - Not necessarily unique to open-source, but lower barrier of entry than paid IP
- Be careful with licensing and patents
  - If you want to sell your invention, check the license
  - Open-source doesn't mean free from patents – could be sued if you sell it

Modularize

Chip level
+
Block level
parallel design



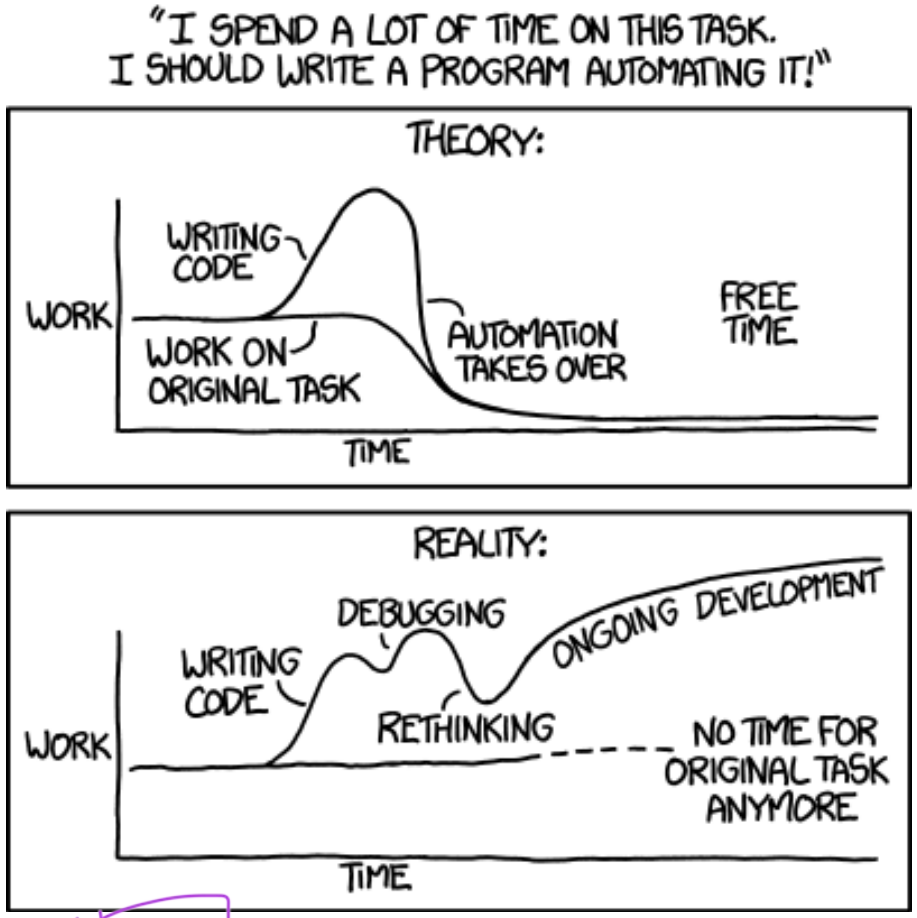
- Create standard interfaces for modules so that they can talk to each other
- **Blackbox**: a portion of your design which you don't know how it works (yet), but you know what the interface and behavior is
- Blackboxing can be used in early design stages
  - Ex: Floorplan at top level, but none of the blocks are ready
  - Solution: make a SWAG at the interface, area, power, etc. and create a blackbox representation of the block
  - Update the blackbox continuously until final implementation
  - Enables *asynchronous* work. Top-level is not (completely) blocked by lower level
  - Enables early investigation and prototyping at a system level

Automate

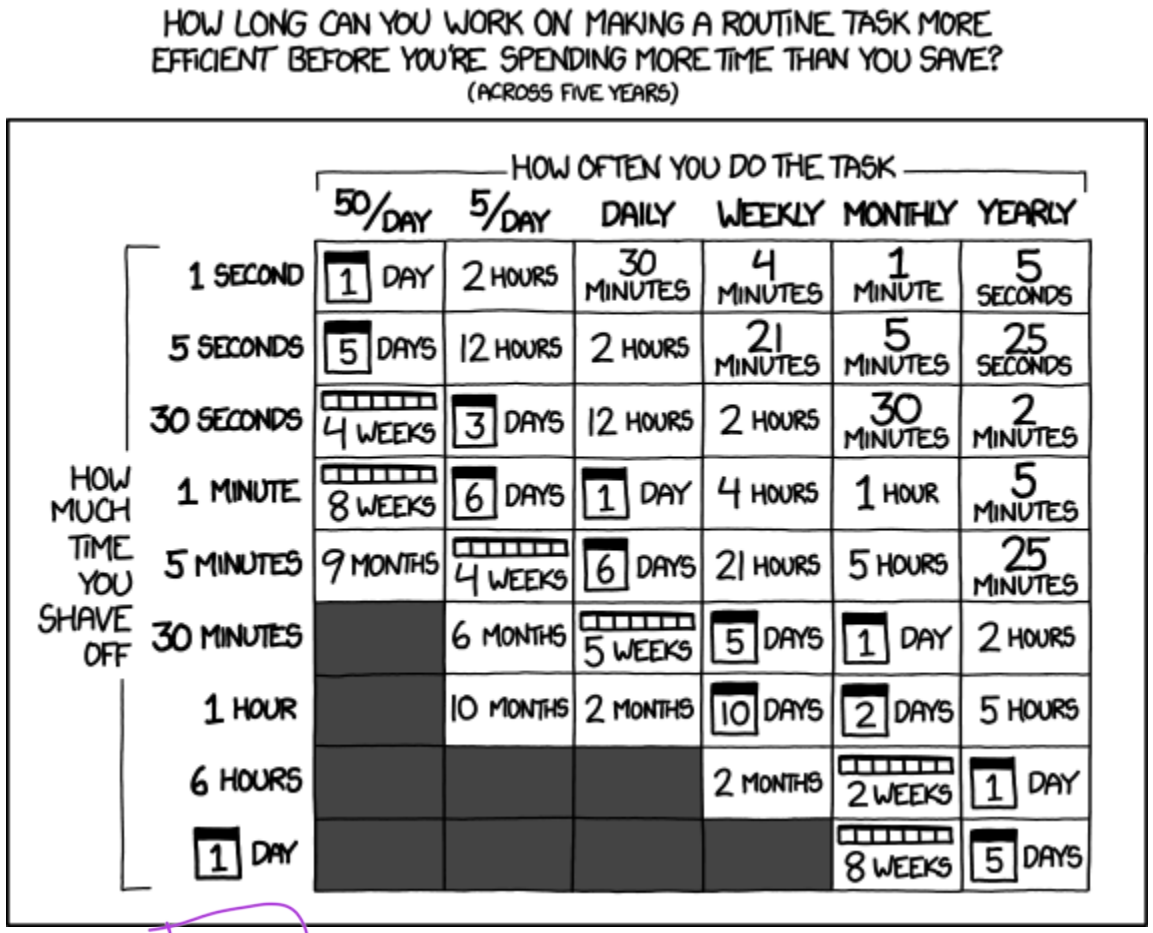
- If you want to be a better HW engineer, think like a SW engineer
- Let the tools do the work for you!!!
- Things to automate: → Cache silent error.
  - Linting/syntax checking: check that your code isn't broken / has common bugs
  - Regression tests: check that your code compiles / passes basic verification
  - Possibly any task that needs to be done more than once!
- Tools for automation
  - Scripting languages: Tcl, Python, Bash, SKILL, etc.
  - APIs to your EDA tools (Innovus, ICC2, OpenROAD, etc.)

charging RTL see how transistor count changes.

Automation Considerations



XKCD 1319 (c) Randall Munroe



XKCD 1205 (c) Randall Munroe

CI / CD

- **Continuous Integration**:
- **Continuous Deployment**:

AWS Code pipeline

Travis CI

Jenkins.

Github actions.

Aside: Technical Debt

- There is often a solution which is “not pretty” but “works”
  - Think of some examples?
- What makes a solution “not pretty”?
  - Not scalable / only works in certain cases
  - Brittle – will break with small changes
  - Not easy to maintain
  - Breaks common design paradigms
- **Technical debt** is incurred from pursuing fast, unsustainable solutions
- When faced with deadlines, technical debt invariably creeps in
- The longevity of your product depends on mitigating technical debt

Technical debt

Takeaways

- Start small, then continuously build on it (MVP)
- Be familiar with your tools, and use all the tools at your disposal
- Don’t reinvent the wheel – use/adapt existing solutions if possible
- Create well-defined interfaces between your components
- Always think about if there’s a way to automate a task

Acknowledgement

- S. Beamer, Class Lecture, Topic: “Course Intro.” CSE 228A, School of Electrical and Computer Engineering, University of California – Santa Cruz, Santa Cruz, CA, 2023.